Java 單位換算器程式碼解析

https://gemini.google.com/app/6e41a7e8324d9c49

User prompt: 告訴我解題步驟以及程式碼

Response: 這是一道使用 Java Swing 製作「單位換算器」的 GUI 程式設計題目。以下為解題步驟與完整程式碼：

解題步驟

1. 建立主視窗與版面配置：
   - 繼承 JFrame 建立視窗，設定大小為 480 x 280，並採用 BorderLayout 作為主要版面配置。
2. 設計 **NORTH** 區域（換算類型）：
   - 建立一個 JComboBox<String>，放入「長度」、「重量」、「溫度」三個選項，放置在 BorderLayout.NORTH。
3. 設計 **CENTER** 區域（數值輸入與單位選擇）：
   - 建立一個使用 GridLayout(2, 1)（兩列）的 JPanel，放入 BorderLayout.CENTER。
   - 第一列放置「來源數值輸入框（JTextField）」與「來源單位下拉選單（JComboBox）」。
   - 第二列放置「目標數值顯示框（JTextField，建議設為不可編輯）」與「目標單位下拉選單（JComboBox）」。
4. 設計 **SOUTH** 區域（執行按鈕）：
   - 建立一個「換算」按鈕（JButton），放置在 BorderLayout.SOUTH。
5. 實作事件監聽（**Event Listeners**）：
   - 切換類型事件：為頂部的類型選單加上 ActionListener。當選項改變時，清空兩個單位下拉選單，並依據選取的類型填入對應的單位陣列（如：公尺、公分、英吋、英尺等）。
   - 換算按鈕事件：為「換算」按鈕加上 ActionListener。讀取第一個 JTextField 的數字（需用 try-catch 處理非數字輸入錯誤），透過「先轉為基準單位（如公尺、公斤、攝氏），再轉為目標單位」的兩段式演算法計算結果，最後輸出至第二個 JTextField。

完整 Java 程式碼

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class UnitConverter extends JFrame {
    // 介面元件
    private JComboBox<String> typeComboBox;
    private JTextField sourceField;
    private JComboBox<String> sourceUnitComboBox;
    private JTextField targetField;
    private JComboBox<String> targetUnitComboBox;
```

```
private JButton convertButton;

// 單位選項資料
private final String[] types = {"長度", "重量", "溫度"};
private final String[] lengthUnits = {"公尺", "公分", "英吋", "英尺"};
private final String[] weightUnits = {"公斤", "公克", "磅", "盎司"};
private final String[] tempUnits = {"攝氏", "華氏", "克氏"};

public UnitConverter() {
    // 1. 視窗基本設定
    setTitle("單位換算器");
    setSize(480, 280);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null); // 視窗置中
    setLayout(new BorderLayout(10, 10));

    // 2. NORTH：選擇換算類型
    typeComboBox = new JComboBox<>(types);
    JPanel northPanel = new JPanel();
    northPanel.add(new JLabel("換算類型："));
    northPanel.add(typeComboBox);
    add(northPanel, BorderLayout.NORTH);

    // 3. CENTER：兩列（來源單位、目標單位）
    JPanel centerPanel = new JPanel(new GridLayout(2, 1, 5, 5));

    // 第一列：來源
    JPanel row1 = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 15));
    sourceField = new JTextField(12);
    sourceUnitComboBox = new JComboBox<>(lengthUnits); // 預設為長度
    row1.add(new JLabel("來源："));
    row1.add(sourceField);
    row1.add(sourceUnitComboBox);

    // 第二列：目標
    JPanel row2 = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 15));
    targetField = new JTextField(12);
    targetField.setEditable(false); // 結果欄位設為唯讀
    targetUnitComboBox = new JComboBox<>(lengthUnits); // 預設為長度
    row2.add(new JLabel("目標："));
    row2.add(targetField);
    row2.add(targetUnitComboBox);

    centerPanel.add(row1);
    centerPanel.add(row2);
    add(centerPanel, BorderLayout.CENTER);

    // 4. SOUTH：「換算」按鈕
    convertButton = new JButton("換算");
    JPanel southPanel = new JPanel();
    southPanel.add(convertButton);
    add(southPanel, BorderLayout.SOUTH);
```

```
        // 5. 事件綁定：切換換算類型時，更新兩個單位下拉選單
        typeComboBox.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                updateUnitComboBoxes();
            }
        });

        // 6. 事件綁定：點擊換算按鈕時計算結果
        convertButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                performConversion();
            }
        });
    }

    // 更新單位下拉選單選項
    private void updateUnitComboBoxes() {
        String selectedType = (String) typeComboBox.getSelectedItem();
        String[] unitsToSet;

        if ("長度".equals(selectedType)) {
            unitsToSet = lengthUnits;
        } else if ("重量".equals(selectedType)) {
            unitsToSet = weightUnits;
        } else {
            unitsToSet = tempUnits;
        }

        // 重新設定模型並清空輸入輸出框
        sourceUnitComboBox.setModel(new DefaultComboBoxModel<>(unitsToSet));
        targetUnitComboBox.setModel(new DefaultComboBoxModel<>(unitsToSet));
        sourceField.setText("");
        targetField.setText("");
    }

    // 執行單位換算邏輯
    private void performConversion() {
        try {
            double inputValue =
Double.parseDouble(sourceField.getText().trim());
            String type = (String) typeComboBox.getSelectedItem();
            String fromUnit = (String) sourceUnitComboBox.getSelectedItem();
            String toUnit = (String) targetUnitComboBox.getSelectedItem();
            double result = 0.0;

            if ("長度".equals(type)) {
                result = convertLength(inputValue, fromUnit, toUnit);
            } else if ("重量".equals(type)) {
                result = convertWeight(inputValue, fromUnit, toUnit);
```

```
            } else if ("溫度".equals(type)) {
                result = convertTemperature(inputValue, fromUnit, toUnit);
            }

            // 格式化輸出至第二個 JTextField（最多顯示 4 位小數並去除多餘的 0）
            targetField.setText(String.format("%.4f", result).replaceAll("0*$",
"").replaceAll("\\.$", ""));
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "請在來源欄位輸入有效的數字！", "輸
入錯誤", JOptionPane.ERROR_MESSAGE);
        }
    }

    // 長度換算（以「公尺」為基準）
    private double convertLength(double val, String from, String to) {
        double meters = switch (from) {
            case "公分" -> val / 100.0;
            case "英吋" -> val * 0.0254;
            case "英尺" -> val * 0.3048;
            default -> val; // 公尺
        };
        return switch (to) {
            case "公分" -> meters * 100.0;
            case "英吋" -> meters / 0.0254;
            case "英尺" -> meters / 0.3048;
            default -> meters; // 公尺
        };
    }

    // 重量換算（以「公斤」為基準）
    private double convertWeight(double val, String from, String to) {
        double kg = switch (from) {
            case "公克" -> val / 1000.0;
            case "磅" -> val * 0.45359237;
            case "盎司" -> val * 0.02834952;
            default -> val; // 公斤
        };
        return switch (to) {
            case "公克" -> kg * 1000.0;
            case "磅" -> kg / 0.45359237;
            case "盎司" -> kg / 0.02834952;
            default -> kg; // 公斤
        };
    }

    // 溫度換算（以「攝氏」為基準）
    private double convertTemperature(double val, String from, String to) {
        double celsius = switch (from) {
            case "華氏" -> (val - 32.0) * 5.0 / 9.0;
            case "克氏" -> val - 273.15;
            default -> val; // 攝氏
        };
```

```
        return switch (to) {
            case "華氏" -> celsius * 9.0 / 5.0 + 32.0;
            case "克氏" -> celsius + 273.15;
            default -> celsius; // 攝氏
        };
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            new UnitConverter().setVisible(true);
        });
    }
}
```

程式設計重點說明

- 基準單位轉換法：如果直接寫「兩兩配對」的換算公式（例如：英吋轉英尺、英尺轉公分），每種分類會有 4×4=16 種組合；改用「來源單位 → 基準單位 → 目標單位」的兩段式寫法，不僅公式清晰不易出錯，程式碼也大幅精簡。
- **Java** 版本相容性：程式碼中的 switch 表達式需要 **Java 14** 以上版本編譯。若您的開發環境為較舊的 Java 8，可將 switch 表達式改寫為傳統的 switch-case 或 if-else 判斷即可。

User prompt: 告訴我怎麼改的

Response: 若要在 **Java 8**（或更早版本）環境下編譯，只需要把原本使用箭頭 -> 的 **switch** 表達式（Java 14 新語法），改回傳統的 **switch-case-break** 寫法即可。

修改對照說明

以 convertLength 為例，修改前後的差異如下：

- 原本寫法（**Java 14+**）：直接用 switch 回傳數值，使用 -> 符號。
```
double meters = switch (from) {
    case "公分" -> val / 100.0;
    default -> val;
};
```
- 修改後寫法（**Java 8** 相容）：先宣告變數，再進入傳統 switch 區塊賦值並加上 break;。
```
double meters;
switch (from) {
    case "公分":
        meters = val / 100.0;
        break;
    default:
        meters = val;
        break;
}
```

修改後的三個換算方法(直接替換原程式碼這三段即可)

你可以將原本程式碼最下方的 convertLength、convertWeight、convertTemperature 三個方法，直接替換成以下寫法：

```
// 長度換算（以「公尺」為基準 - Java 8 相容寫法）
private double convertLength(double val, String from, String to) {
    double meters;
    switch (from) {
        case "公分":
            meters = val / 100.0;
            break;
        case "英吋":
            meters = val * 0.0254;
            break;
        case "英尺":
            meters = val * 0.3048;
            break;
        default: // 公尺
            meters = val;
            break;
    }

    switch (to) {
        case "公分":
            return meters * 100.0;
        case "英吋":
            return meters / 0.0254;
        case "英尺":
            return meters / 0.3048;
        default: // 公尺
            return meters;
    }
}

// 重量換算（以「公斤」為基準 - Java 8 相容寫法）
private double convertWeight(double val, String from, String to) {
    double kg;
    switch (from) {
        case "公克":
            kg = val / 1000.0;
            break;
        case "磅":
            kg = val * 0.45359237;
            break;
        case "盎司":
            kg = val * 0.02834952;
            break;
        default: // 公斤
            kg = val;
```

```
            break;
        }

        switch (to) {
            case "公克":
                return kg * 1000.0;
            case "磅":
                return kg / 0.45359237;
            case "盎司":
                return kg / 0.02834952;
            default: // 公斤
                return kg;
        }
    }

    // 溫度換算（以「攝氏」為基準 - Java 8 相容寫法）
    private double convertTemperature(double val, String from, String to) {
        double celsius;
        switch (from) {
            case "華氏":
                celsius = (val - 32.0) * 5.0 / 9.0;
                break;
            case "克氏":
                celsius = val - 273.15;
                break;
            default: // 攝氏
                celsius = val;
                break;
        }

        switch (to) {
            case "華氏":
                return celsius * 9.0 / 5.0 + 32.0;
            case "克氏":
                return celsius + 273.15;
            default: // 攝氏
                return celsius;
        }
    }
```

小細節提醒

- 第一段 switch (from) 因為需要繼續往下執行第二段轉換，所以每個 case 結尾必須加上 **break;**，否則會繼續往下執行覆蓋掉變數值。
- 第二段 switch (to) 因為直接用 **return** 回傳結果並結束方法，所以不需要再寫 break;。
- 程式碼中的 Lambda 表達式（例如 SwingUtilities.invokeLater(() -> ...)）在 **Java 8** 就已經正式支援，因此不需要修改。

User prompt: 給我完整程式碼

Response: 以下為已整合 **Java 8** 相容寫法的完整程式碼，直接複製並儲存為 UnitConverter.java 即可編譯執行：

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class UnitConverter extends JFrame {
    // 介面元件
    private JComboBox<String> typeComboBox;
    private JTextField sourceField;
    private JComboBox<String> sourceUnitComboBox;
    private JTextField targetField;
    private JComboBox<String> targetUnitComboBox;
    private JButton convertButton;

    // 單位選項資料
    private final String[] types = {"長度", "重量", "溫度"};
    private final String[] lengthUnits = {"公尺", "公分", "英吋", "英尺"};
    private final String[] weightUnits = {"公斤", "公克", "磅", "盎司"};
    private final String[] tempUnits = {"攝氏", "華氏", "克氏"};

    public UnitConverter() {
        // 1. 視窗基本設定 (480x280, BorderLayout)
        setTitle("單位換算器");
        setSize(480, 280);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null); // 視窗置中
        setLayout(new BorderLayout(10, 10));

        // 2. NORTH：JComboBox 選換算類型
        typeComboBox = new JComboBox<>(types);
        JPanel northPanel = new JPanel();
        northPanel.add(new JLabel("換算類型："));
        northPanel.add(typeComboBox);
        add(northPanel, BorderLayout.NORTH);

        // 3. CENTER：兩列（來源單位、目標單位）
        JPanel centerPanel = new JPanel(new GridLayout(2, 1, 5, 5));

        // 第一列：來源
        JPanel row1 = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 15));
        sourceField = new JTextField(12);
        sourceUnitComboBox = new JComboBox<>(lengthUnits); // 預設為長度
        row1.add(new JLabel("來源："));
        row1.add(sourceField);
        row1.add(sourceUnitComboBox);

        // 第二列：目標
        JPanel row2 = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 15));
        targetField = new JTextField(12);
```

```
        targetField.setEditable(false); // 結果欄位設為唯讀
        targetUnitComboBox = new JComboBox<>(lengthUnits); // 預設為長度
        row2.add(new JLabel("目標："));
        row2.add(targetField);
        row2.add(targetUnitComboBox);

        centerPanel.add(row1);
        centerPanel.add(row2);
        add(centerPanel, BorderLayout.CENTER);

        // 4. SOUTH：「換算」按鈕
        convertButton = new JButton("換算");
        JPanel southPanel = new JPanel();
        southPanel.add(convertButton);
        add(southPanel, BorderLayout.SOUTH);

        // 5. 事件綁定：切換換算類型時，更新兩個單位下拉選單
        typeComboBox.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                updateUnitComboBoxes();
            }
        });

        // 6. 事件綁定：點擊換算按鈕時計算結果
        convertButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                performConversion();
            }
        });
    }

    // 更新單位下拉選單選項
    private void updateUnitComboBoxes() {
        String selectedType = (String) typeComboBox.getSelectedItem();
        String[] unitsToSet;

        if ("長度".equals(selectedType)) {
            unitsToSet = lengthUnits;
        } else if ("重量".equals(selectedType)) {
            unitsToSet = weightUnits;
        } else {
            unitsToSet = tempUnits;
        }

        // 重新設定選項模型並清空輸入輸出框
        sourceUnitComboBox.setModel(new DefaultComboBoxModel<>(unitsToSet));
        targetUnitComboBox.setModel(new DefaultComboBoxModel<>(unitsToSet));
        sourceField.setText("");
        targetField.setText("");
    }
```

```
    // 執行單位換算邏輯
    private void performConversion() {
        try {
            double inputValue =
Double.parseDouble(sourceField.getText().trim());
            String type = (String) typeComboBox.getSelectedItem();
            String fromUnit = (String) sourceUnitComboBox.getSelectedItem();
            String toUnit = (String) targetUnitComboBox.getSelectedItem();
            double result = 0.0;

            if ("長度".equals(type)) {
                result = convertLength(inputValue, fromUnit, toUnit);
            } else if ("重量".equals(type)) {
                result = convertWeight(inputValue, fromUnit, toUnit);
            } else if ("溫度".equals(type)) {
                result = convertTemperature(inputValue, fromUnit, toUnit);
            }

            // 格式化輸出至第二個 JTextField（最多顯示 4 位小數並去除多餘的 0）
            targetField.setText(String.format("%.4f", result).replaceAll("0*$",
"").replaceAll("\\.$", ""));
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "請在來源欄位輸入有效的數字！", "輸
入錯誤", JOptionPane.ERROR_MESSAGE);
        }
    }

    // 長度換算（以「公尺」為基準 - Java 8 相容寫法）
    private double convertLength(double val, String from, String to) {
        double meters;
        switch (from) {
            case "公分":
                meters = val / 100.0;
                break;
            case "英吋":
                meters = val * 0.0254;
                break;
            case "英尺":
                meters = val * 0.3048;
                break;
            default: // 公尺
                meters = val;
                break;
        }

        switch (to) {
            case "公分":
                return meters * 100.0;
            case "英吋":
                return meters / 0.0254;
            case "英尺":
```

```
            return meters / 0.3048;
        default: // 公尺
            return meters;
    }
}

// 重量換算（以「公斤」為基準 - Java 8 相容寫法）
private double convertWeight(double val, String from, String to) {
    double kg;
    switch (from) {
        case "公克":
            kg = val / 1000.0;
            break;
        case "磅":
            kg = val * 0.45359237;
            break;
        case "盎司":
            kg = val * 0.02834952;
            break;
        default: // 公斤
            kg = val;
            break;
    }

    switch (to) {
        case "公克":
            return kg * 1000.0;
        case "磅":
            return kg / 0.45359237;
        case "盎司":
            return kg / 0.02834952;
        default: // 公斤
            return kg;
    }
}

// 溫度換算（以「攝氏」為基準 - Java 8 相容寫法）
private double convertTemperature(double val, String from, String to) {
    double celsius;
    switch (from) {
        case "華氏":
            celsius = (val - 32.0) * 5.0 / 9.0;
            break;
        case "克氏":
            celsius = val - 273.15;
            break;
        default: // 攝氏
            celsius = val;
            break;
    }

    switch (to) {
```

```
            case "華氏":
                return celsius * 9.0 / 5.0 + 32.0;
            case "克氏":
                return celsius + 273.15;
            default: // 攝氏
                return celsius;
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            new UnitConverter().setVisible(true);
        });
    }
}
```

User prompt: 這是什麼

Response: **Java 8**（正式版本號為 Java SE 8，又稱 JDK 1.8）是 Java 程式語言歷史上最經典且普及率極高的一個長期支援版本（**LTS**），於 2014 年正式發布。

在這個程式設計情境中，特別提到它的原因如下：

- 學校與考試的標準環境：許多大學的「物件導向程式設計」或「視窗程式設計」課程，為了確保教材與舊有實驗室電腦的相容性，至今仍以 **Java 8** 作為標準教學與上機考的環境。
- 語法相容性分水嶺：Java 在後續版本（例如 Java 14、Java 17、Java 21）加入許多更簡潔的新語法（如第一版程式碼中使用的 switch 箭頭表達式 ->）。如果拿新語法的程式碼放到安裝 Java 8 的電腦上執行，編譯器會看不懂並直接報錯（Syntax Error）。
- 向上相容特性：Java 具備「向下相容（Backward Compatibility）」的特性。這代表用 **Java 8** 傳統語法寫出來的程式碼，不僅能在舊版的 Java 8 環境順利執行，放到最新的 Java 21 或更新版本中也完全可以正常編譯與運作。

簡單來說，「Java 8 相容寫法」就是指「使用最穩妥的經典語法，確保你的程式碼在任何新舊電腦或學校實驗室環境中都能順利執行、不會報錯」。

User prompt: 視窗的來源和目標分別要輸入什麼

Response: 在這個「單位換算器」視窗中，來源和目標的操作方式如下：

- 來源（第一列）：
  - 輸入框（**JTextField**）：由你手動輸入想要換算的「原始數字」（例如：輸入 100）。
  - 下拉選單（**JComboBox**）：選擇這個數字原本的單位（例如：選「公分」）。
- 目標（第二列）：
  - 輸入框（**JTextField**）：不需要輸入（程式碼已將它設為唯讀、不可打字），點擊下方的「換算」按鈕後，系統會自動把計算結果顯示在這裡（例如：顯示 1）。
  - 下拉選單（**JComboBox**）：選擇你想要轉換成的目標單位（例如：選「公尺」）。

實際操作範例

假設你想知道 **100** 公分等於幾公尺：

1. 頂部「換算類型」維持選取 長度。
2. 第一列「來源」：左邊格子輸入 **100**，右邊選單選 公分。
3. 第二列「目標」：左邊格子留白不填，右邊選單選 公尺。
4. 點擊最下方的「換算」按鈕，第二列的格子就會自動跑出結果 **1**。
